REDES Y SERVICIOS DE COMUNICACIONES

DEPARTAMENTO DE INGENIERIA DE SISTEMAS Y COMPUTACION

Laboratorio #3:

Análisis capa de trasporte y sockets

Grupo 10:

Rodrigo Garcia Aguirre

Fabio Enrique Higuera Pacanchique

Nicolas David Valentin Avendaño

Contenido

1. Ejecución de Publicadores y Subscriptores 3

1.1 TCP 3

1.2 UDP 4

2. Comparativa TCP y UDP 7

3. Preguntas de Análisis 8

•3.1 ¿Qué ocurriría si en lugar de dos publicadores (partidos transmitidos) hubiera cien partidos simultáneos? ¿Cómo impactaría esto en el desempeño del bróker bajo TCP y bajo UDP? 8

• 3. 2 Si un gol se envía como mensaje desde el publicador y un suscriptor no lo recibe en UDP, ¿qué implicaciones tendría para la aplicación real? ¿Por qué TCP maneja mejor este escenario? 8

3.3 En un escenario de seguimiento en vivo de partidos, ¿qué protocolo (TCP o UDP) resultaría más adecuado? Justifique con base en los resultados de la práctica. 9

3.4 Compare el overhead observado en las capturas Wireshark entre TCP y UDP. ¿Cuál protocolo introduce más cabeceras por mensaje? ¿Cómo influye esto en la eficiencia? 9

3.5 Si el marcador de un partido llega desordenado en UDP (por ejemplo, primero se recibe el 2–1 y luego el 1–1), ¿qué efectos tendría en la experiencia del usuario? ¿Cómo podría solucionarse este problema a nivel de aplicación? 9

3.6¿Cómo cambia el desempeño del sistema cuando aumenta el número de suscriptores interesados en un mismo partido? ¿Qué diferencias se observaron entre TCP y UDP en este aspecto? 10

3.7 ¿Qué sucede si el bróker se detiene inesperadamente? ¿Qué diferencias hay entre TCP y UDP en la capacidad de recuperación de la sesión? 10

3.8 ¿Cómo garantizar que todos los suscriptores reciban en el mismo instante las actualizaciones críticas (por ejemplo, un gol)? ¿Qué protocolo facilita mejor esta sincronización y por qué? 10

3.9 Analice el uso de CPU y memoria en el broker cuando maneja múltiples conexiones TCP frente al manejo de datagramas UDP. ¿Qué diferencias encontró? 10

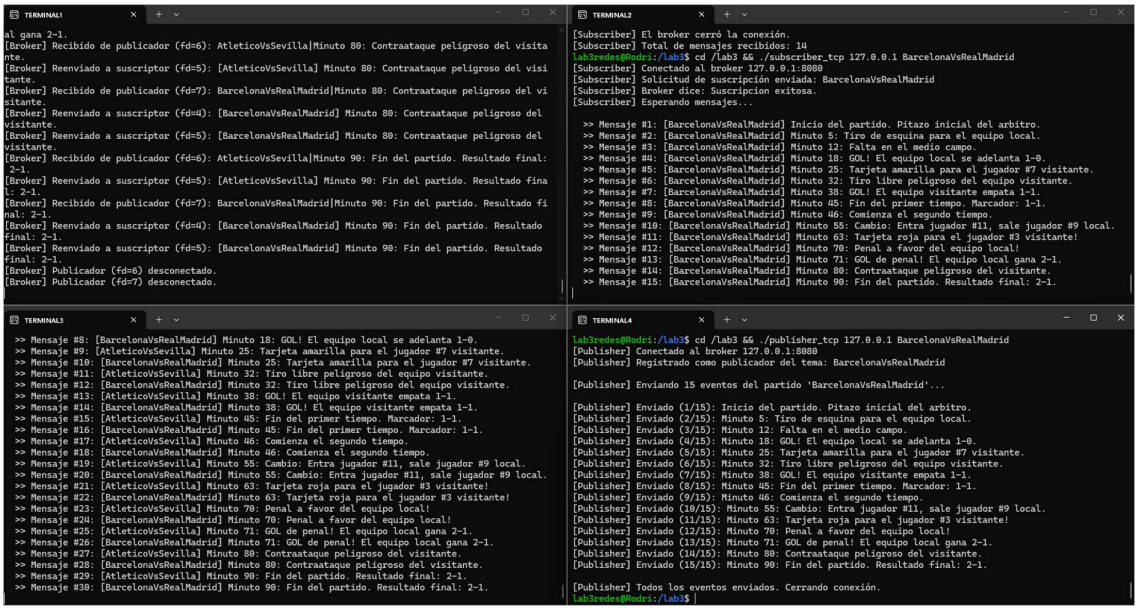
3.10 Si tuviera que diseñar un sistema real de transmisión de actualizaciones de partidos de fútbol para millones de usuarios, ¿elegiría TCP, UDP o una combinación de ambos? Justifique con base en lo observado en el laboratorio 11

1. Ejecución de Publicadores y Subscriptores

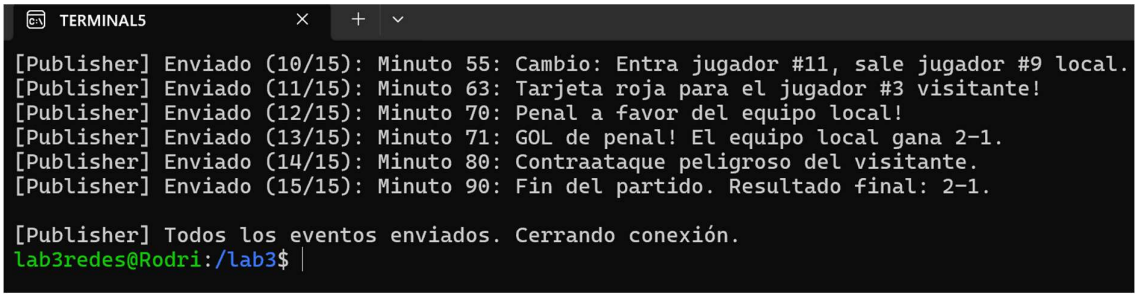
Para este laboratorio se quiere identificar la diferencia de hacer un envió de paquetes con TCP o con UDP, Para esto se programaron y ejecutaron, varias instancias de publicadores y suscriptores, que simularan él envió de datos de lo que sucede en distintos partidos, a los diferentes suscriptores interesados en cada uno de estos.

1.1 TCP

Inicialmente se probó el protocolo de transferencia TCP. Al correrlo en la consola WSL se pueden ver que se envían y llegan correctamente los paquetes a los diferentes suscriptores.



1.Terminales WSL TCP 1



2.Terminales WSL TCP 2

Al capturar el tráfico con wireshark logramos ver que se envían y llegan correctamente todos los paquetes, tomando un tiempo aproximado de 67 segundos para él envió.

146	59.737242	127.0.0.1	127.0.0.1	TCP	66	45374 → 8080 [ACK] Seq=26 Ack=897 Win=65536 Len=0 TSval=1075773738 TSecr=1075773738
147	59.737286	127.0.0.1	127.0.0.1	TCP	133	8080 → 38984 [PSH, ACK] Seq=1589 Ack=44 Win=65536 Len=67 TSval=1075773738 TSecr=1075773495
148	59.737343	127.0.0.1	127.0.0.1	TCP	66	38984 → 8080 [ACK] Seq=44 Ack=1656 Win=65536 Len=0 TSval=1075773738 TSecr=1075773738
149	61.494685	127.0.0.1	127.0.0.1	TCP	135	54460 → 8080 [PSH, ACK] Seq=825 Ack=1 Win=65536 Len=69 TSval=1075775495 TSecr=1075773495
150	61.494721	127.0.0.1	127.0.0.1	TCP	66	8080 → 54460 [ACK] Seq=1 Ack=894 Win=65536 Len=0 TSval=1075775495 TSecr=1075775495
151	61.494877	127.0.0.1	127.0.0.1	TCP	137	8080 → 38984 [PSH, ACK] Seq=1656 Ack=44 Win=65536 Len=71 TSval=1075775495 TSecr=1075773738
152	61.494935	127.0.0.1	127.0.0.1	TCP	66	38984 → 8080 [ACK] Seq=44 Ack=1727 Win=65536 Len=0 TSval=1075775495 TSecr=1075775495
153	61.737352	127.0.0.1	127.0.0.1	TCP	139	54474 → 8080 [PSH, ACK] Seq=877 Ack=1 Win=65536 Len=73 TSval=1075775738 TSecr=1075773738
154	61.737450	127.0.0.1	127.0.0.1	TCP	66	8080 → 54474 [ACK] Seq=1 Ack=950 Win=65536 Len=0 TSval=1075775738 TSecr=1075775738
155	61.737588	127.0.0.1	127.0.0.1	TCP	141	8080 → 45374 [PSH, ACK] Seq=897 Ack=26 Win=65536 Len=75 TSval=1075775738 TSecr=1075773738
156	61.737696	127.0.0.1	127.0.0.1	TCP	66	45374 → 8080 [ACK] Seq=26 Ack=972 Win=65536 Len=0 TSval=1075775738 TSecr=1075775738
157	61.737732	127.0.0.1	127.0.0.1	TCP	141	8080 → 38984 [PSH, ACK] Seq=1727 Ack=44 Win=65536 Len=75 TSval=1075775738 TSecr=1075775495
158	61.737787	127.0.0.1	127.0.0.1	TCP	66	38984 → 8080 [ACK] Seq=44 Ack=1802 Win=65536 Len=0 TSval=1075775738 TSecr=1075775738
159	62.756486	127.0.0.1	127.0.0.1	TCP	133	54460 → 8080 [PSH, ACK] Seq=894 Ack=1 Win=65536 Len=67 TSval=1075777499 TSecr=1075775495
160	62.756527	127.0.0.1	127.0.0.1	TCP	66	8080 → 54460 [ACK] Seq=1 Ack=961 Win=65536 Len=0 TSval=1075777499 TSecr=1075777499
161	62.756586	127.0.0.1	127.0.0.1	TCP	135	8080 → 38984 [PSH, ACK] Seq=1802 Ack=44 Win=65536 Len=69 TSval=1075777499 TSecr=1075775738
162	62.756622	127.0.0.1	127.0.0.1	TCP	66	38984 → 8080 [ACK] Seq=44 Ack=1871 Win=65536 Len=0 TSval=1075777499 TSecr=1075777499
163	63.039595	127.0.0.1	127.0.0.1	TCP	137	54474 → 8080 [PSH, ACK] Seq=950 Ack=1 Win=65536 Len=71 TSval=1075777782 TSecr=1075775738
164	63.039640	127.0.0.1	127.0.0.1	TCP	66	8080 → 54474 [ACK] Seq=1 Ack=1021 Win=65536 Len=0 TSval=1075777782 TSecr=1075777782
165	63.039711	127.0.0.1	127.0.0.1	TCP	139	8080 → 45374 [PSH, ACK] Seq=972 Ack=26 Win=65536 Len=73 TSval=1075777783 TSecr=1075775738
166	63.039747	127.0.0.1	127.0.0.1	TCP	66	45374 → 8080 [ACK] Seq=26 Ack=1045 Win=65536 Len=0 TSval=1075777783 TSecr=1075777783
167	63.039769	127.0.0.1	127.0.0.1	TCP	139	8080 → 38984 [PSH, ACK] Seq=1871 Ack=44 Win=65536 Len=73 TSval=1075777783 TSecr=1075777499
168	63.039801	127.0.0.1	127.0.0.1	TCP	66	38984 → 8080 [ACK] Seq=44 Ack=1944 Win=65536 Len=0 TSval=1075777783 TSecr=1075777783
169	64.766504	127.0.0.1	127.0.0.1	TCP	134	54460 → 8080 [PSH, ACK] Seq=961 Ack=1 Win=65536 Len=68 TSval=1075779509 TSecr=1075777499
170	64.766545	127.0.0.1	127.0.0.1	TCP	66	8080 → 54460 [ACK] Seq=1 Ack=1029 Win=65536 Len=0 TSval=1075779509 TSecr=1075779509
171	64.766660	127.0.0.1	127.0.0.1	TCP	136	8080 → 38984 [PSH, ACK] Seq=1944 Ack=44 Win=65536 Len=70 TSval=1075779509 TSecr=1075777783
172	64.766725	127.0.0.1	127.0.0.1	TCP	66	38984 → 8080 [ACK] Seq=44 Ack=2014 Win=65536 Len=0 TSval=1075779510 TSecr=1075779509
173	65.040288	127.0.0.1	127.0.0.1	TCP	138	54474 → 8080 [PSH, ACK] Seq=1021 Ack=1 Win=65536 Len=72 TSval=1075779783 TSecr=1075777782
174	65.040355	127.0.0.1	127.0.0.1	TCP	66	8080 → 54474 [ACK] Seq=1 Ack=1093 Win=65536 Len=0 TSval=1075779783 TSecr=1075779783
175	65.040463	127.0.0.1	127.0.0.1	TCP	140	8080 → 45374 [PSH, ACK] Seq=1045 Ack=26 Win=65536 Len=74 TSval=1075779783 TSecr=1075777783
176	65.040534	127.0.0.1	127.0.0.1	TCP	66	45374 → 8080 [ACK] Seq=26 Ack=1119 Win=65536 Len=0 TSval=1075779783 TSecr=1075779783
177	65.040572	127.0.0.1	127.0.0.1	TCP	140	8080 → 38984 [PSH, ACK] Seq=2014 Ack=44 Win=65536 Len=74 TSval=1075779783 TSecr=1075779510
178	65.040632	127.0.0.1	127.0.0.1	TCP	66	38984 → 8080 [ACK] Seq=44 Ack=2088 Win=65536 Len=0 TSval=1075779783 TSecr=1075779783
179	66.766869	127.0.0.1	127.0.0.1	TCP	66	54460 → 8080 [FIN, ACK] Seq=1029 Ack=1 Win=65536 Len=0 TSval=1075781510 TSecr=1075779509
180	66.766990	127.0.0.1	127.0.0.1	TCP	66	8080 → 54460 [FIN, ACK] Seq=1 Ack=1030 Win=65536 Len=0 TSval=1075781510 TSecr=1075781510
181	66.767097	127.0.0.1	127.0.0.1	TCP	66	54460 → 8080 [ACK] Seq=1030 Ack=2 Win=65536 Len=0 TSval=1075781510 TSecr=1075781510
182	67.040660	127.0.0.1	127.0.0.1	TCP	66	54474 → 8080 [FIN, ACK] Seq=1093 Ack=1 Win=65536 Len=0 TSval=1075781784 TSecr=1075779783
183	67.040850	127.0.0.1	127.0.0.1	TCP	66	8080 → 54474 [FIN, ACK] Seq=1 Ack=1094 Win=65536 Len=0 TSval=1075781784 TSecr=1075781784
184	67.040985	127.0.0.1	127.0.0.1	TCP	66	54474 → 8080 [ACK] Seq=1094 Ack=2 Win=65536 Len=0 TSval=1075781784 TSecr=1075781784

3.Captura de Trafico Wireshark TCP

Al anializar los paquetes se puede apreciar que se envia un paquete de 132 bytes en total, con payload de 66 bytes, indicando que este tiene un overhead de aproximadamente 66 bytes, esto dado a que se añaden varios campos adicionales, como lo puede ser ack o 32 bytes de header, que controlan el orden y la integridad de los datos, a la vez que realiza el manejo de confirmaciones, para que los paquetes lleguen de manera correcta al destino.

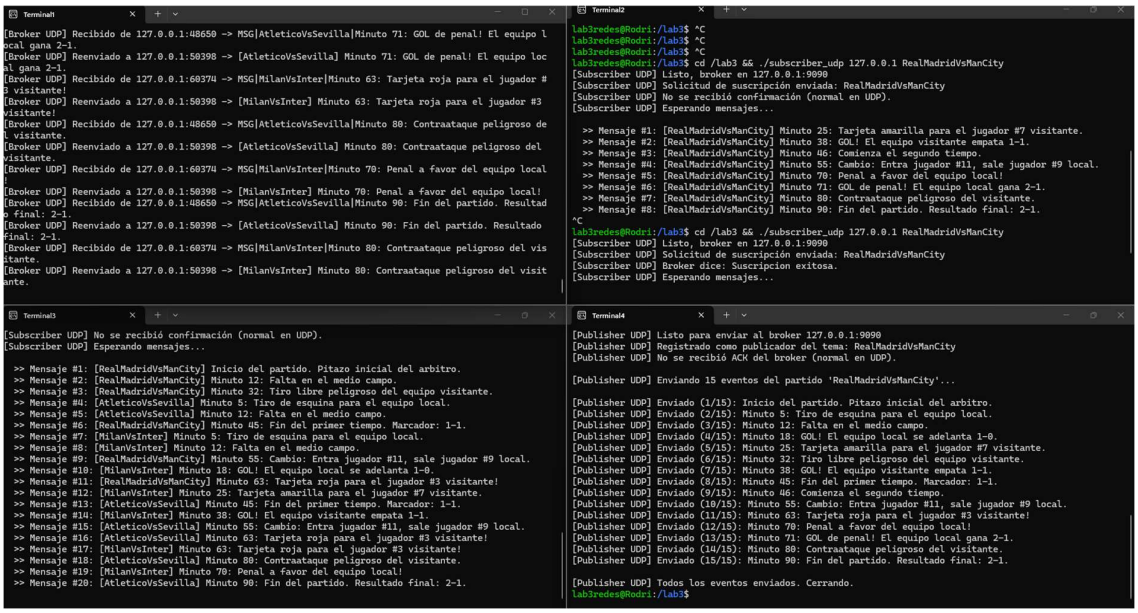
```
▸ Frame 26: Packet, 132 bytes on wire (1056 bits), 132 bytes captured (1056 bits)
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 54460, Dst Port: 8080, Seq: 22, Ack: 1, Len: 66
    Source Port: 54460
    Destination Port: 8080
    [Stream index: 4]
    [Stream Packet Number: 6]
  ▸ [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 66]
    Sequence Number: 22    (relative sequence number)
    Sequence Number (raw): 2460055535
    [Next Sequence Number: 88    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 3613109394
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x018 (PSH, ACK)
    Window: 512
    [Calculated window size: 65536]
    [Window size scaling factor: 128]
    Checksum: 0xfe6a [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  ▸ [Timestamps]
  ▸ [SEQ/ACK analysis]
    [Client Contiguous Streams: 1]
    [Server Contiguous Streams: 1]
    TCP payload (66 bytes)
    TCP segment data (66 bytes)
▸ [2 Reassembled TCP Segments (87 bytes): #24(21), #26(66)]

0000  00 00 00 00 00 00 00 00  00 00 00 00 08 00 45 00   ........ ......E.
0010  00 76 39 c1 40 00 40 06  02 bf 7f 00 00 01 7f 00   .v9.@.@. ........
0020  00 01 d4 bc 1f 90 92 a1  77 ef d7 5b ac 92 80 18   ........ w..[....
0030  02 00 fe 6a 00 00 01 01  08 0a 40 1e a9 fa 40 1e   ...j.... ..@...@.
0040  a9 fa 41 74 6c 65 74 69  63 6f 56 73 53 65 76 69   ..Atleti coVsSevi
0050  6c 6c 61 7c 49 6e 69 63  69 6f 20 64 65 6c 20 70   lla|Inic io del p
0060  61 72 74 69 64 6f 2e 20  50 69 74 61 7a 6f 20 69   artido.  Pitazo i
0070  6e 69 63 69 61 6c 20 64  65 6c 20 61 72 62 69 74   nicial d el arbit
0080  72 6f 2e 0a                                         ro.·
```

4.Overhead TCP

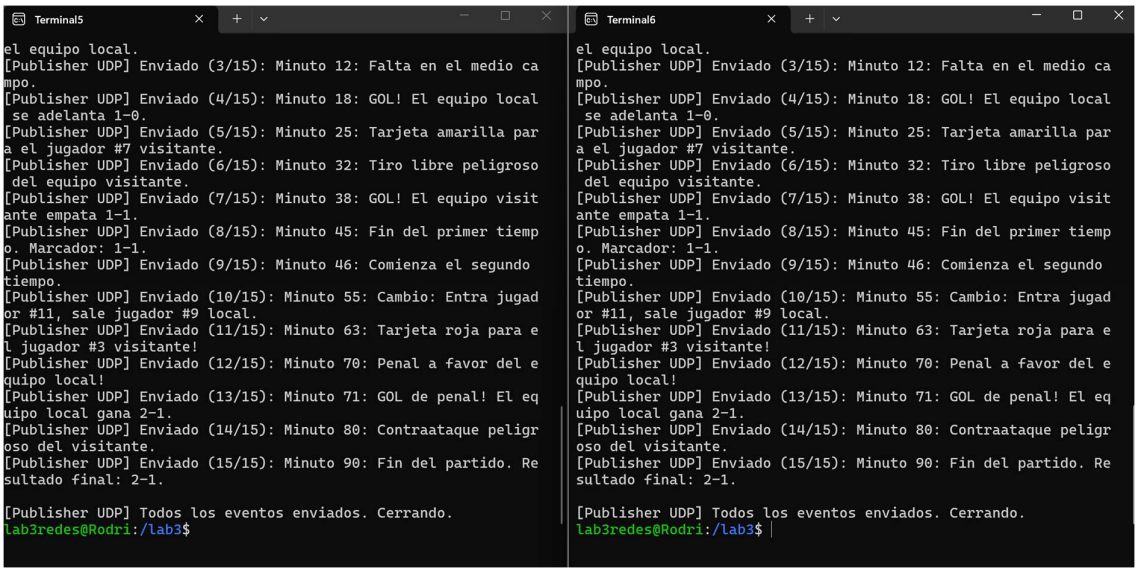
1.2 UDP

De igual manera se probo el protocolo UDP, donde al correrlo se evidencia claramente que a pesar de que los publicadores envían una cantidad de mensajes, en este caso 15

cada publicador, los suscriptores, pierden algunos mensajes y no obtienen toda la información, que buscan.



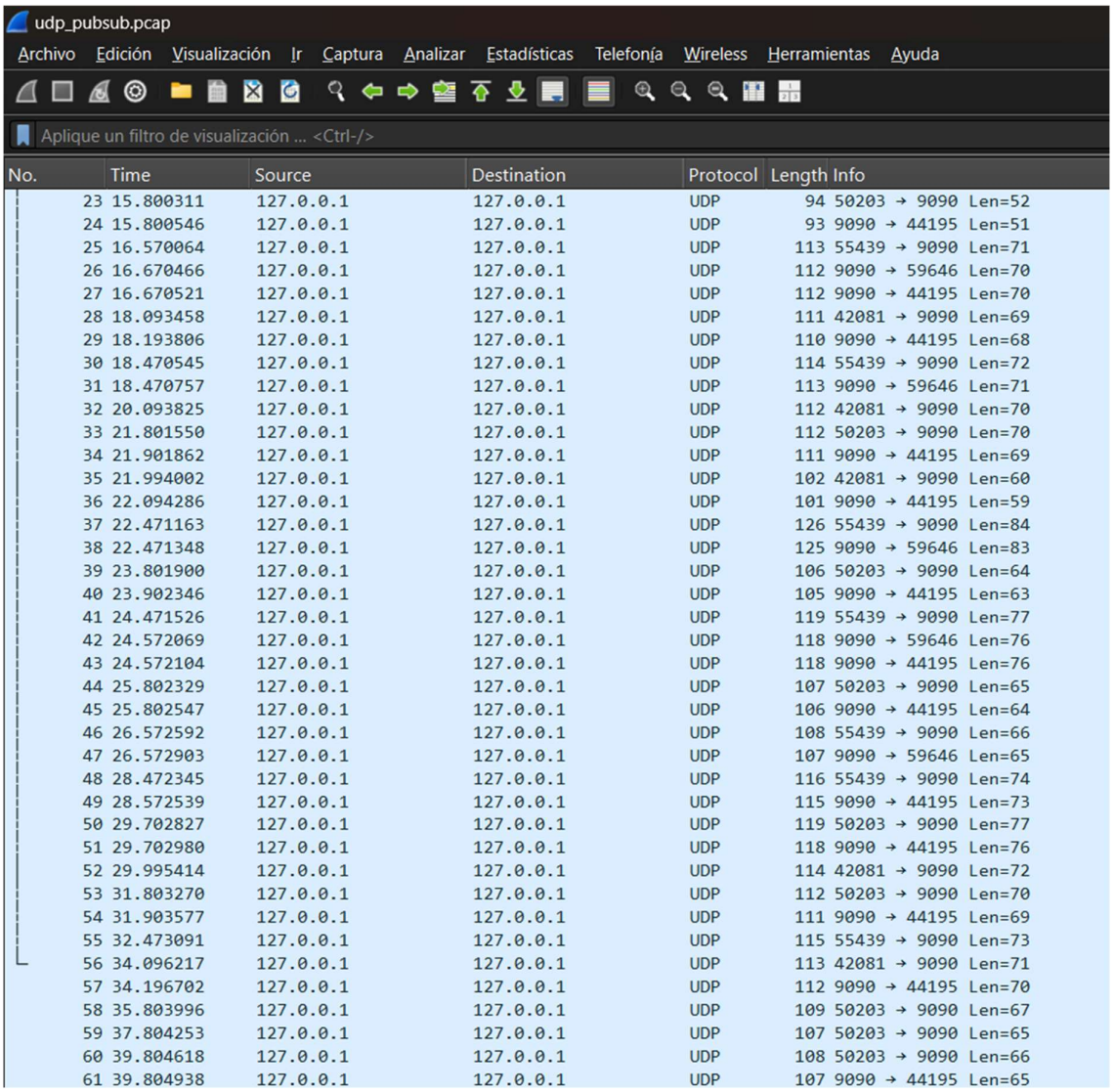
5.Terminales WSL UDP 1



6.Terminales WSL UDP 2

El análisis de Wireshark se identificó que el broker recibió tres mensajes UDP consecutivos de MilanVsInter, visibles en los paquetes 58, 59 y 60, con destino al puerto 9090. Pero vemos que solo se registró un reenvío posterior hacia el suscriptor en el paquete 61, correspondiente al evento del minuto 90. Esto indica que, aunque los mensajes de los minutos 71 y 80 fueron recibidos por el broker, no se evidencia su entrega al suscriptor dentro de la captura. Por tanto, la traza muestra una pérdida de mensajes en la comunicación UDP. Al finalizar el proceso se aprecia que UDP, tardo

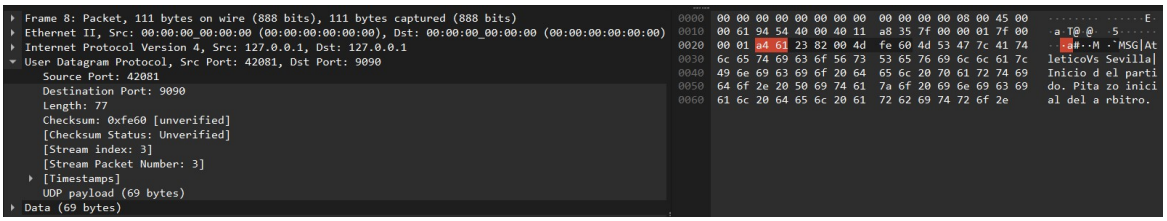
apenas poco más de 39 segundos en terminar el envío de paquetes, al no realizar las confirmaciones.



No.	Time	Source	Destination	Protocol	Length	Info
23	15.800311	127.0.0.1	127.0.0.1	UDP	94	50203 → 9090 Len=52
24	15.800546	127.0.0.1	127.0.0.1	UDP	93	9090 → 44195 Len=51
25	16.570064	127.0.0.1	127.0.0.1	UDP	113	55439 → 9090 Len=71
26	16.670466	127.0.0.1	127.0.0.1	UDP	112	9090 → 59646 Len=70
27	16.670521	127.0.0.1	127.0.0.1	UDP	112	9090 → 44195 Len=70
28	18.093458	127.0.0.1	127.0.0.1	UDP	111	42081 → 9090 Len=69
29	18.193806	127.0.0.1	127.0.0.1	UDP	110	9090 → 44195 Len=68
30	18.470545	127.0.0.1	127.0.0.1	UDP	114	55439 → 9090 Len=72
31	18.470757	127.0.0.1	127.0.0.1	UDP	113	9090 → 59646 Len=71
32	20.093825	127.0.0.1	127.0.0.1	UDP	112	42081 → 9090 Len=70
33	21.801550	127.0.0.1	127.0.0.1	UDP	112	50203 → 9090 Len=70
34	21.901862	127.0.0.1	127.0.0.1	UDP	111	9090 → 44195 Len=69
35	21.994002	127.0.0.1	127.0.0.1	UDP	102	42081 → 9090 Len=60
36	22.094286	127.0.0.1	127.0.0.1	UDP	101	9090 → 44195 Len=59
37	22.471163	127.0.0.1	127.0.0.1	UDP	126	55439 → 9090 Len=84
38	22.471348	127.0.0.1	127.0.0.1	UDP	125	9090 → 59646 Len=83
39	23.801900	127.0.0.1	127.0.0.1	UDP	106	50203 → 9090 Len=64
40	23.902346	127.0.0.1	127.0.0.1	UDP	105	9090 → 44195 Len=63
41	24.471526	127.0.0.1	127.0.0.1	UDP	119	55439 → 9090 Len=77
42	24.572069	127.0.0.1	127.0.0.1	UDP	118	9090 → 59646 Len=76
43	24.572104	127.0.0.1	127.0.0.1	UDP	118	9090 → 44195 Len=76
44	25.802329	127.0.0.1	127.0.0.1	UDP	107	50203 → 9090 Len=65
45	25.802547	127.0.0.1	127.0.0.1	UDP	106	9090 → 44195 Len=64
46	26.572592	127.0.0.1	127.0.0.1	UDP	108	55439 → 9090 Len=66
47	26.572903	127.0.0.1	127.0.0.1	UDP	107	9090 → 59646 Len=65
48	28.472345	127.0.0.1	127.0.0.1	UDP	116	55439 → 9090 Len=74
49	28.572539	127.0.0.1	127.0.0.1	UDP	115	9090 → 44195 Len=73
50	29.702827	127.0.0.1	127.0.0.1	UDP	119	50203 → 9090 Len=77
51	29.702980	127.0.0.1	127.0.0.1	UDP	118	9090 → 44195 Len=76
52	29.995414	127.0.0.1	127.0.0.1	UDP	114	42081 → 9090 Len=72
53	31.803270	127.0.0.1	127.0.0.1	UDP	112	50203 → 9090 Len=70
54	31.903577	127.0.0.1	127.0.0.1	UDP	111	9090 → 44195 Len=69
55	32.473091	127.0.0.1	127.0.0.1	UDP	115	55439 → 9090 Len=73
56	34.096217	127.0.0.1	127.0.0.1	UDP	113	42081 → 9090 Len=71
57	34.196702	127.0.0.1	127.0.0.1	UDP	112	9090 → 44195 Len=70
58	35.803996	127.0.0.1	127.0.0.1	UDP	109	50203 → 9090 Len=67
59	37.804253	127.0.0.1	127.0.0.1	UDP	107	50203 → 9090 Len=65
60	39.804618	127.0.0.1	127.0.0.1	UDP	108	50203 → 9090 Len=66
61	39.804938	127.0.0.1	127.0.0.1	UDP	107	9090 → 44195 Len=65

7.Trafico Wirsehark UDP

En cuanto al análisis de los paquetes, se observa que se tiene un paquete de 11 bytes, para el mismo mensaje que se dio en TCP, con un overhead de aproximadamente 42 bytes, siendo menor, dado a que la estructura de los paquetes en mas simple, contando con solo campos básicos como los puertos de origen y destino, omitiendo números de secuencia, o control de flujo que hacen que a pesar de tener una carga menor, no se garantice la integridad de los datos.



8.Overhead UDP

Enlace repositorio GitHub:
https://github.com/Rodri45/lab3_pubsub

2. Comparativa TCP y UDP

A partir de los resultados del laboratorio, se realizó la siguiente comparación entre los protocolos UDP y TCP.

	TCP	**UDP**
Perdida de paquetes	En TCP no se aprecia una perdida de paquetes, todos los paquetes que se envían, llegan al suscriptor de manera correcta, ya que se envían confirmaciones de que los mensajes fueron recibidos correctamente.	En UDP, se pierden varios paquetes de los que fueron enviados por el Publisher. No detecta si se perdieron mensajes, pues no existe una confirmación de la llegada de los mensajes. Ademas, al manejarse los datagramas de manera independiente, los paquetes podrían no llegar en el orden en que se enviaron.
Tiempo	Según las pruebas realizadas TCP tarda unos 67 segundos en realizar él envió completo de los paquetes, lo que a pesar de tardar más que UDP, dado a que realiza confirmaciones, asegura la llegada e integridad de estos.	Según las pruebas realizadas, UDP, tarda alrededor, de unos 39 segundos en terminar el envío de los paquetes, lo cual a pesar de ser más rápido que lo que hace TCP, al no confirmar los mensajes, se puede dar el caso de que se pierdan paquetes.
Overhead	En las pruebas realizadas se observa que TCP tiene un Overhead de aproximadamente 66	En las pruebas realizadas, UDP muestra tener un Ovehead de aproximadamente 42

	bytes, en donde se incluyen datos relevantes para verificar el orden y integridad de los datos gracias a las confirmaciones.	bytes, en donde el ahorro que se tiene de carga, es el no incluir la información relevante para mantener la integridad de los datos, lo que puede causar que se pierda información.

En base a los resultados, se puede concluir, que a pesar de que en UDP se pueda tener mejor rendimiento y ser mas rápido, no es muy confiable si lo que se busca es mantener la integridad de los datos, en donde TCP seria una mejor alternativa, ya que a pesar de tardar mas tiempo y tener más carga, se asegura de que los datos lleguen de manera correcta.

3. Preguntas de Análisis

•3.1 ¿Qué ocurriría si en lugar de dos publicadores (partidos transmitidos) hubiera cien partidos simultáneos? ¿Cómo impactaría esto en el desempeño del bróker bajo TCP y bajo UDP?

En un escenario con TCP el bróker tendría que manejar muchas conexiones simultaneas, una por publicador y varias por suscriptor, además aumenta el uso de CPU y memoria por el manejo de estados, buffers y control de flujo, y por esto puede tender a volverse lento o a saturarse.

Para un caso con UDP no existen como tal las conexiones, entonces el bróker solo recibe y envía datagramas, por esto mismo es más ligero, pero aumenta el riesgo de pérdida de mensajes como se observó en el desarrollo del laboratorio.

• 3. 2 Si un gol se envía como mensaje desde el publicador y un suscriptor no lo recibe en UDP, ¿qué implicaciones tendría para la aplicación real? ¿Por qué TCP maneja mejor este escenario?

Si esto pasa en una aplicacion real el usuario nunca se enteraría de la ocurrencia del gol y por ende la experiencia sería mala e inconsistente, TCP manejaría esto mejor porque tiene retrasmisión automática con lo cual garantiza que lleguen todos los paquetes sin perdidas y además de esto que lleguen en orden.

3.3 En un escenario de seguimiento en vivo de partidos, ¿qué protocolo (TCP o UDP) resultaría más adecuado? Justifique con base en los resultados de la práctica.

Si se necesita evaluar o escoger solo uno de los dos protocolos, se tendría que ver las necesidades puntuales, ya que de acuerdo con los resultados nos pudimos dar cuenta que si se quiere ver en vivo con mejor respuesta en tiempo real y menor latencia y se permite cierta perdida de paquetes lo mejor es UDP, y si se quiere que no se pierda ningún evento sería mejor TCP ya que nos asegura más precisión y confiabilidad. En base a esto creemos que la mejor opción es TCP ya que asegura que el cliente recibe los paquetes completos a pesar de que lleguen con algo de retraso, ya que con esto no se perderían eventos de los partidos ni abrían inconsistencias con los mismos.

3.4 Compare el overhead observado en las capturas Wireshark entre TCP y UDP. ¿Cuál protocolo introduce más cabeceras por mensaje? ¿Cómo influye esto en la eficiencia?

En las capturas de Wireshark se observa que TCP introduce más overhead que UDP. Aunque ambos paquetes transportan una cantidad similar de datos útiles, el paquete TCP tiene un tamaño total mayor de 132 bytes con 66 bytes de payload que el paquete UDP que tiene 111 bytes con 69 bytes de payload. Esto ocurre porque TCP añade más campos de control, como números de secuencia, entre otros, mientras que UDP solo incluye una cabecera. Por lo que se puede concluir que UDP es más eficiente en uso de ancho de banda y rapidez, mientras que TCP sacrifica eficiencia para ofrecer mayor confiabilidad y control en la transmisión

3.5 Si el marcador de un partido llega desordenado en UDP (por ejemplo, primero se recibe el 2–1 y luego el 1–1), ¿qué efectos tendría en la experiencia del usuario? ¿Cómo podría solucionarse este problema a nivel de aplicación?

Los efectos o problemas que generaría esto para estos casos seria que la información seria inconsistente, lo cual generaría confusión en el usuario y por ende una mala experiencia de usuario.

Unas posibles soluciones podrían ser agregar timestamps o usar números de secuencia, además también se podría reordenar los mensajes en el cliente.

3.6¿Cómo cambia el desempeño del sistema cuando aumenta el número de suscriptores interesados en un mismo partido? ¿Qué diferencias se observaron entre TCP y UDP en este aspecto?

El desempeño del sistema se ve afectado principalmente en el broker ya que debe distribuir más mensajes simultáneamente. En TCP, el impacto es mayor ya que se mantiene una conexión independiente con cada suscriptor y esto usa más CPU y memoria. Por lo cual al aumentar suscriptores el sistema se vuelve más lento y saturarse. En cambio, en UDP el broker no maneja conexiones, solo envía datagramas lo que hace que el consumo de recursos sea menor y el envío más rápido, pero al aumentar suscriptores con UDP puede aumentar también la perdida de paquetes.

3.7 ¿Qué sucede si el bróker se detiene inesperadamente? ¿Qué diferencias hay entre TCP y UDP en la capacidad de recuperación de la sesión?

Con TCP lo que pasaría es que las conexiones se cerrarían, los clientes deberían reconectar y además se perdería la sesión. Por otro lado, con UDP no hay conexiones por lo cual solo se dejan de recibir mensajes y es más fácil reanudar la sesión.

3.8 ¿Cómo garantizar que todos los suscriptores reciban en el mismo instante las actualizaciones críticas (por ejemplo, un gol)? ¿Qué protocolo facilita mejor esta sincronización y por qué?

Garantizar que las actualizaciones o mensajes les lleguen a todos en el mismo instante es muy complejo por las variaciones de red, pero esto se puede intentar lograr con técnicas como multicast, timestamps y buffers de sincronización. El protocolo que facilitaría mejor esto será UDP, ya que no requiere establecer conexiones ni esperar confirmaciones, lo que reduce la latencia. Además, permite enviar los mensajes casi simultáneamente a los clientes, lo cual con TCP no se logra ya que lo hace de forma secuencial por conexión, por lo cual sería mejor opción UDP, aunque este no garantiza la entrega de todos los paquetes lo ideal son los críticos y evita retrasos de retrasmisión y control de congestión y con esto sería más sencillo lograr las actualizaciones críticas en el mismo instante para los clientes.

3.9 Analice el uso de CPU y memoria en el broker cuando maneja múltiples conexiones TCP frente al manejo de datagramas UDP. ¿Qué diferencias encontró?

El broker con TCP usa más CPU y memoria porque debe mantener múltiples conexiones activas, estados, buffers y control para la confiabilidad. En cambio, UDP

consume menos ya que no hay conexiones ni control de estados, solo manejo de datagramas. Por lo cual, TCP es más costoso en base a uso de CPU y memoria, pero más confiable y UDP es más eficiente y escalable.

3.10 Si tuviera que diseñar un sistema real de transmisión de actualizaciones de partidos de fútbol para millones de usuarios, ¿elegiría TCP, UDP o una combinación de ambos? Justifique con base en lo observado en el laboratorio

Lo mejor sería una combinación entre TCP y UDP, esto porque, por un lado, se utilizaría UDP para enviar las actualizaciones en tiempo real, ya que ofrece menor latencia, mayor escalabilidad y permite distribuir la información a muchos usuarios simultáneamente, esto para la rapidez de respuesta. Por otro lado, TCP serviría para el manejo de aspectos o eventos críticos como registro de usuarios, recuperación de sesión o de información pérdida importante y sincronización, ya que nos garantiza la entrega correcta y ordenada de los datos. Con base en lo observado en el laboratorio, TCP nos mostró la confiabilidad de la que hablamos, pero con mayor overhead y menor eficiencia en escenarios masivos, mientras que UDP nos mostró mayor rapidez. Por ello una forma combinada de estos dos protocolos nos permite un mejor equilibrio entre rendimiento, confiablidad y escalabilidad para un sistema de gran escala.